

C Programming Lab

Introduction to C Syntax

Learning Objectives

By the end of this lab, students should be able to:

- Write and compile a basic C program.
- Declare and initialize variables.
- Read different types of input from the keyboard.
- Print formatted output.
- Use arithmetic and relational operators.
- Implement if, else if, else, and switch statements.
- Define and call functions (both with and without return values).
- Use arrays.
- Implement for, while, and do-while loops.
- Understand basic C syntax and its differences from Python.

Before we start programming with c, have a look on the most common header we include in C programming and the corresponding functions.

Header	Main purpose	Common functions / features
<code><stdio.h></code>	Input / Output	printf(), scanf(), fgets(), fopen(), fclose(), fprintf()
<code><stdlib.h></code>	General utilities	malloc(), calloc(), realloc(), free(), atoi(), rand(), exit()
<code><string.h></code>	String manipulation	strlen(), strcpy(), strcat(), strcmp(), strstr(), memcpy()
<code><math.h></code>	Mathematical operations	sqrt(), pow(), sin(), cos(), log(), ceil(), floor()
<code><ctype.h></code>	Character testing/conversion	isalpha(), isdigit(), isalnum(), isspace(), toupper(), tolower()
<code><time.h></code>	Date and time	time(), clock(), localtime(), strftime()
<code><stdbool.h></code>	Boolean values	bool, true, false
<code><stdint.h></code>	Fixed-width integers	int8_t, uint8_t, int32_t, uint64_t
<code><limits.h></code>	Integer limits	INT_MAX, INT_MIN, CHAR_BIT, LONG_MAX
<code><float.h></code>	Floating-point limits	FLT_MAX, DBL_MAX, FLT_EPSILON
<code><assert.h></code>	Program assertions/testing	assert()
<code><errno.h></code>	Error handling	errno
<code><stddef.h></code>	Common types/constants	size_t, NULL, ptrdiff_t

<stdarg.h>	Variable arguments	va_list, va_start(), va_arg(), va_end()
<conio.h>	Console I/O (<i>non-standard</i>)	getch(), kbhit()

Python **import** VS C **#include**

C: #include is → a text/preprocessing mechanism → the C **preprocessor** essentially makes the declarations from stdio.h available to the compiler before compilation.

Python: import is a runtime/module mechanism → Python does **not** simply copy the contents of imported library into your program.

1. Input data → scanf() ← reading input from keyboard and write it in memory address

Part 1 – Examples (Instructor Demonstration)

Example 1 — Read Different Types of Input

Write a C program that reads different primitive data types from the keyboard and displays their values.

```
#include <stdio.h>
int main() {
    int age;
    float height;
    double salary;
    char grade;
    char name[50];

    //Input
    printf("Enter your name: ");
    scanf("%49s", name);           // string → %s

    printf("Enter your age: ");
    scanf("%d", &age);             // int → %d

    printf("Enter your height: ");
    scanf("%f", &height);          //float → %f

    printf("Enter your salary: ");
    scanf("%lf", &salary);         //double → %lf ← scanf / and / %f or %lf ← printf

    printf("Enter your grade: ");
    scanf(" %c", &grade);          // char → %c

    //Output
    printf("\n--- Student Information ---\n");
```

```
printf("Name: %s\n", name);
printf("Age: %d\n", age);
printf("Height: %.2f\n", height);
printf("Salary: %.2lf\n", salary);
printf("Grade: %c\n", grade);

return 0;
}
```

Explanation

- C requires explicit type declaration for every variable.
- Compare with Python:

Python → age = 20

C → int age = 20;

Important: The & in scanf("%d", &age) is required by C's input mechanism. We will explain why when we cover pointers. For now, treat it as part of the scanf() syntax.

Exercise 1 – Personal Data

Write a program that asks the user for:

- Their favourite integer number,
- Their favourite decimal number (float),
- Their favourite letter.

Use scanf() to read the input, and then print each value back with a descriptive message.

2. Printing → printf()

Example 2 : Practice formatted output using printf().

```
#include <stdio.h>

int main() {

    int number = 42;
```

```

float price = 19.95f;
double pi = 3.1415926535;
char grade = 'A';
char name[] = "Anas";

printf("Integer: %d\n", number);
printf("Float: %f\n", price);
printf("Float with 2 decimals: %.2f\n", price);
printf("Double: %.4f\n", pi);
printf("Character: %c\n", grade);
printf("String: %s\n", name);

printf("\n--- Formatted Output ---\n");

printf("Number = %10d\n", number); //use 10 character positions
printf("Price = %10.2f\n", price); //use 10 characters 5 of them will be take for the 5 places/characters
                                number 19.95
printf("Name = %-10s\n", name); //this will left align the name in 10 character positions from the left side of
                                the screen, and fill the remaining space with spaces
printf("Name = %s\n", name);    //this will print the name without any formatting, it will take as much space as
                                needed for the name

//the different between %-10s and %s can be shown clearly in following example
printf("|%s|\n", name);
printf("|%-10s|\n", name);

return 0;
}

```

Compare with **Python's** f-strings: `print(f"{price:.2f}")`!

Exercise 2 – Shopping Bill

Write a program that:

- Declares variables for price (float), quantity (int), and total (float).
- Reads price and quantity from the user.
- Computes `total = price * quantity`.
- Prints the result in a formatted table like this:

Item	Price	Qty	Total
Book	19.95	3	59.85

Use printf() with width specifiers to align the columns.

Example 3 — Arithmetic Operations

Objective

Use arithmetic operators with variables.

```
#include <stdio.h>

int main() {

    int a = 20;
    int b = 6;

    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b); // integer division! → because in c 20 / 6 gives 3, not 3.333
    printf("%.2f\n", (float)a / b);
    printf("a %% b = %d\n", a % b);

    return 0;
}
```

Exercise 3 – All Operations

Write a program that:

- Reads two integers from the user.
- Calculates and stores: sum, difference, product, quotient (as a float), and remainder.
- Prints each result on a separate line with a description.

Requirement: Perform all calculations in separate variables (not directly inside printf()).

3. Control Statements

Example 4 — Conditional Statements

Implement decision-making using if, else if, and else.

```

#include <stdio.h>

int main() {

    int grade;

    printf("Enter your grade: ");
    scanf("%d", &grade);

    if (grade >= 90) {
        printf("Grade: A\n");
    }
    else if (grade >= 80) {
        printf("Grade: B\n");
    }
    else if (grade >= 70) {
        printf("Grade: C\n");
    }
    else if (grade >= 60) {
        printf("Grade: D\n");
    }
    else {
        printf("Grade: F\n");
    }

    return 0;
}

```

Python → C Comparison

Python

C

if condition:

if (condition) {

elif condition:

else if (condition) {

else:

else {

indentation

{ } and ;

Exercise 4 – Age Classifier

Write a program that reads a person's age and prints:

- "Child" if age < 13
- "Teenager" if age between 13 and 19 (inclusive)
- "Adult" if age between 20 and 64
- "Senior" if age ≥ 65

Example 5 — Logical and Relational Operators

Use conditions involving multiple expressions.

```
#include <stdio.h>
```

```
int main() {
```

```
    int age;  
    int hasID;
```

```
    printf("Enter your age: ");  
    scanf("%d", &age);
```

```
    printf("Do you have an ID? (1 = Yes, 0 = No): ");  
    scanf("%d", &hasID);
```

```
    if (age >= 18 && hasID == 1) {  
        printf("Access granted.\n");  
    }  
    else {  
        printf("Access denied.\n");  
    }
```

```
    return 0;  
}
```

Operators

Meaning	C Symbol
Equal	==
Not equal	!=

Meaning	C Symbol
Greater than	>
Less than	<
Greater/equal	>=
Less/equal	<=
AND	&&
OR	
NOT	!

Python uses → **and, or, not** – while – C uses → **&&, ||, !**.

Exercise 5 – Triangle Validity

Write a program that reads three side lengths (integers) and determines if they form a valid triangle. A triangle is valid if the sum of any two sides is greater than the third side. Print "Valid triangle" or "Invalid triangle".

Example 6 — switch Statement

Implement multiple choices using switch.

```
#include <stdio.h>

int main() {

    int option;

    printf("1. Add\n");
```

```

printf("2. Subtract\n");
printf("3. Multiply\n");
printf("4. Divide\n");

printf("Choose an option: ");
scanf("%d", &option);

switch (option) {

    case 1:
        printf("You selected Add.\n");
        break;

    case 2:
        printf("You selected Subtract.\n");
        break;

    case 3:
        printf("You selected Multiply.\n");
        break;

    case 4:
        printf("You selected Divide.\n");
        break;

    default:
        printf("Invalid option.\n");
}

return 0;
}

```

Important: The break statement prevents execution from “falling through” to the next case.

Exercise 6 – Simple Calculator

Write a program using switch that:

- Reads two numbers and an operator (+, -, *, /) as a character.
- Performs the corresponding operation and prints the result.
- If the operator is '/', check if the second number is zero – if so, print "Division by zero!".
- Use switch on the operator character.

Hint: For the operator, use char op; scanf(" %c", &op); (note the space before %c to skip whitespace).

Example 7 — Define and Call Functions

Objective

Define functions, pass arguments, and return values.

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}

int main() {

    int x = 10;
    int y = 5;

    int sum = add(x, y);
    int product = multiply(x, y);

    printf("Sum = %d\n", sum);
    printf("Product = %d\n", product);

    return 0;
}
```

Function Structure

```
return_type function_name(parameters) {

    // statements

    return value;

}
```

Example: `int add(int a, int b)` – returns an integer and takes two integers.

Python Comparison

Python	C
<code>def add(a, b):</code>	<code>int add(int a, int b) {</code>
<code>return a + b</code>	<code>return a + b;</code>
No type specifiers	Types required for parameters and return

Example 8 — Function Without a Return Value

```
#include <stdio.h>
```

```
void printWelcome() {  
    printf("Welcome to C programming!\n");  
}
```

```
void printLine() {  
    printf("-----\n");  
}
```

```
int main() {
```

```
    printLine();  
    printWelcome();  
    printLine();
```

```
    return 0;  
}
```

void means the function does not return a value.

Exercise 7 – Max and Min

Write a program with two functions:

- `int max(int a, int b)` – returns the larger of two integers.
- `int min(int a, int b)` – returns the smaller of two integers.

In `main()`, read two numbers from the user and print both the maximum and minimum using these functions.

Exercise 8 – Heading and Footer

Write a program that:

- Defines a void printHeading() function that prints "=== RESULTS ===".
- Defines a void printFooter() function that prints "=== END ===".
- In main(), read two numbers, compute their sum in a separate function (named sum), and display the result with the heading and footer around it.

Note: The sum function should take two integers and return an integer.

Example 9 — for Loop

Objective

Implement repetition using a for loop.

```
#include <stdio.h>
```

```
int main() {  
  
    for (int i = 1; i <= 5; i++) {  
        printf("Iteration %d\n", i);  
    }  
  
    return 0;  
}
```

Python Comparison

Python

C

for i in range(1, 6):

for (int i = 1; i <= 5; i++) {

print(i)

printf("%d\n", i);

C does not have a built-in range() function; the loop is explicitly controlled by initialization, condition, and increment.

Example 10 — while Loop

Use a while loop to repeat an operation while a condition is true.

Syntax

```
while (condition) {  
    // statements  
}  
  
#include <stdio.h>  
int main() {  
    int number = 1;  
  
    while (number <= 5) {  
        printf("%d\n", number);  
        number++;  
    }  
  
    return 0;  
}
```

Example 11 — do-while Loop

Objective

Understand the difference between while and do-while.

```
#include <stdio.h>  
  
int main() {  
    int number;  
  
    do {  
        printf("Enter a positive number: ");  
        scanf("%d", &number);  
    } while (number <= 0);  
  
    printf("You entered: %d\n", number);  
}
```

```
    return 0;
}
```

Important:

A do-while loop executes its body **at least once** because the condition is checked after the body.

Exercise 9 – Multiplication Table

Write a program that:

- Asks the user for a number n.
- Prints the multiplication table for n from 1 to 10 using a for loop.

Sample Output:

Enter a number: 7

7 x 1 = 7

7 x 2 = 14

...

7 x 10 = 70

Exercise 10 – Password Check (while)

Write a program that:

- Continuously asks the user for a password.
- The correct password is "secret" (case-sensitive).
- If the user enters the correct password, print "Access granted" and exit the loop.
- Use a while loop.

Hint: Use `char password[20];` and `strcmp()` from `<string.h>` to compare strings.

Example: if (`strcmp(password, "secret") == 0`)

Exercise 11 – Input Validation (do-while)

Write a program that asks the user to enter a number between 1 and 10 (inclusive).
Re-prompt until the user gives a valid number.
Use a do-while loop.

Example 12 — Arrays

C Introduce fixed-size arrays.

```
#include <stdio.h>

int main() {

    int grades[5];

    printf("Enter 5 grades:\n");

    for (int i = 0; i < 5; i++) {
        scanf("%d", &grades[i]);
    }

    printf("\nGrades:\n");

    for (int i = 0; i < 5; i++) {
        printf("Grade %d = %d\n", i + 1, grades[i]);
    }

    return 0;
}
```

Python Comparison

- Python: `grades = [0] * 5`
- C: `int grades[5];`
- Both use zero-based indexing.

Exercise 12 – Array Statistics

Write a program that:

- Reads 10 integers into an array.
- Finds and prints the **maximum**, **minimum**, and **average** (as a float) of the array elements.
- You can use a single loop to compute sum and track min/max.

Example 13 — Combined Program (Functions, Conditions, Loops)

Create a program that reads three grades, calculates the average, and determines the student's result.

This example reinforces: Variables, Input / Output, Data types, Arithmetic, Functions (parameters & return), Conditional statements, Formatted output.

```
#include <stdio.h>

float calculateAverage(int a, int b, int c) {
    return (a + b + c) / 3.0f;
}

void printResult(float average) {

    if (average >= 90) {
        printf("Grade: A\n");
    }
    else if (average >= 80) {
        printf("Grade: B\n");
    }
    else if (average >= 70) {
        printf("Grade: C\n");
    }
    else if (average >= 60) {
        printf("Grade: D\n");
    }
    else {
        printf("Grade: F\n");
    }
}

int main() {

    int grade1, grade2, grade3;
    float average;

    printf("Enter three grades: ");
    scanf("%d %d %d", &grade1, &grade2, &grade3);

    average = calculateAverage(grade1, grade2, grade3);

    printf("Average: %.2f\n", average);

    printResult(average);
}
```

```
    return 0;  
}
```

Exercise 13 – Complete Grade Report

Write a complete program that:

1. Reads **5 student grades** (use a loop to fill an array).
2. Uses a function `float calculateAverage(int grades[], int size)` to compute the average.
3. Uses a function `char getLetterGrade(float avg)` to return the letter grade (A, B, C, D, F) based on the average.
4. Prints:
 - All grades (one per line)
 - The average (with two decimals)
 - The letter grade

Bonus: Also print the highest and lowest grade.